

CERTOOL — Manual

MGZ Consulting

Table of Contents

CERTOOL

A platform-agnostic, local-web-UI **test-campaign recorder** for the bench. It walks an operator record-by-record through a test's run sheet, forces and reads instrument settings where a device connector supports it (falling back to *smart manual entry* otherwise), and writes a self-describing, organised output folder: an Obsidian report, an xlsx of the calculated tabular data, and timestamped records so bench photos correlate by capture time.

Built by **MGZ Consulting** for CE-marking test campaigns and released as an open-source resource. The tool itself contains **no client-confidential data** — test methods derive from public standards; product data lives only in generated session output.

Highlights

- **One local web UI**, one self-contained binary per platform. Cross-platform (Linux, Windows).
- **Live instrument control** via a bench-4wire connector — Keysight E36312A sources the current, Rigol DM858E reads the drop — or **smart manual entry** when no instrument is attached.
- **Colour status light** (orange ready · green OK · red off-limit · yellow unstable) and a keystroke-driven capture loop (Enter commit, r read).
- **Append-only records** with re-measure (superseded attempts retained), **suspend/resume any session**, and **incremental points** with auto-fill (progressive numbering + repeated presets).
- **Instrument serials auto-captured** from *IDN?; **AZ 88163 ambient logger** export merged by timestamp.
- **Organised output**: campaigns/<EUT>/<test>/<date>/ with report, xlsx, and an evidence/ tree.

Install

See [docs/INSTALL.md](#) (and the PDF manual). In short:

- **Linux**: download the certool-linux package, ./install.sh (binary + udev rule for /dev/usbtmc). No Python/VISA needed.

- **Windows:** download the certool-windows bundle, run install.bat (sets up a local Python venv for the pyvisa bridge), install a VISA runtime (Keysight IO Libraries), then run.bat.

Then open <http://127.0.0.1:8787/>.

Build from source

Requires Go 1.24+.

```
go run ./cmd/certool          # serve http://127.0.0.1:8787
GOOS=windows go build ./cmd/certool # cross-compile for Windows
bash packaging/linux/build.sh    # Linux package
bash packaging/windows/build.sh  # Windows bundle
```

Architecture

Three layers, so new tests and instruments plug in without touching the core:

```
framework (internal/{engine,model,output,web,ambient})
├── session · run sheet · timestamped records · xlsx + Obsidian md · web UI
├── test apps   (internal/testapp/*) ← the "why" + acceptance logic
├── earth-continuity (EN 60204-1 Cl.18) ← reference implementation
├── device connectors (internal/connector/*)
    ├── manual ← default smart-fallback (expected values prefilled)
    └── bench-4wire · keysight · rigol · korad · scpi (USBTMC + LAN + pyvisa bridge)
```

Documentation

- [docs/USER-GUIDE.md](#) — the bench workflow, end to end.
- [docs/INSTALL.md](#) — install for Linux and Windows.

Licence

MIT — see [LICENSE](#).

CERTOOL — installation

CERTOOL is a single local binary that serves a web UI at <http://127.0.0.1:8787/>. Pick your platform.

Linux

On Linux the tool talks to USBTMC instruments **directly via /dev/usbtmc*** — no Python or VISA runtime is needed.

1. Download and unpack the certool-linux-amd64.tar.gz package.
2. Run the installer (installs the binary and a udev rule for instrument access):

```
cd certool-linux
./install.sh      # uses sudo for /usr/local/bin + /etc/udev/rules.d
```

3. One-time: make sure your user is in the plugdev group, then log out/in:

```
sudo usermod -aG plugdev "$USER"
```

4. Run it:

```
certool -out ~/certool-campaigns
```

A browser opens at <http://127.0.0.1:8787/>.

Instruments: plug the Keysight/Rigol in by USB; they appear as /dev/usbtmc0, /dev/usbtmc1, ... and the bench-4wire connector auto-discovers them by *IDN?. If a device stays root-only, re-check the udev rule and the plugdev membership.

Windows

Windows has no built-in USBTMC support, so live instrument control goes through a small **Python/pyvisa bridge** that the installer sets up in a self-contained .venv (nothing is installed system-wide by the tool).

1. Download and unpack the certool-windows bundle to a writable folder (e.g. the Desktop) — **not** a read-only location.
2. Install **Python 3** from python.org — tick *Add Python to PATH*.
3. Run **install.bat** — it creates .venv, installs pyvisa etc., then lists any instruments it can see.
4. Make the instruments visible to VISA — install **Keysight IO Libraries Suite** (free; provides the VISA runtime + USBTMC drivers for *both* the Keysight and Rigol). Re-run install.bat to confirm they list as USB0::0x2A8D:.... / USB0::0x1AB1:..... (*Advanced alternative: bind each instrument to WinUSB with Zadig and use the pyvisa-py fallback — see the bundle README.*)
5. Run **run.bat** → browser opens at <http://127.0.0.1:8787/>.

Running in a VM: SPICE USB redirection can *enumerate* a USBTMC instrument but not sustain its I/O session (VISA VI_ERROR_ALLOC). Use native USB passthrough instead, and give the VM ≥ 8 GiB RAM.

First run

1. Pick the test (e.g. *Protective Bonding Continuity*).
2. Fill the EUT/session fields; choose the **bench-4wire** connector (or manual).
3. Follow [USER-GUIDE.md](#) for the capture workflow, pause/ resume, re-measure, ambient import, and report generation.

Output is written under your -out folder, organised as <EUT>/<test>/<YYYY-MM-DD_HHMMSS>/ with the report, results.xlsx, and an evidence/ tree.

certool — bench user guide

How to run a test campaign end to end: set up, capture, pause/resume on a failure, re-run a point, merge ambient, add photos, and produce the organised report folder.

1. Start the tool

- **Linux (dev/bench):** go run ./cmd/certool -out <output-folder>
- **Windows (lab):** run run.bat (after install.bat + a VISA runtime — see the bundle README). A browser opens at <http://127.0.0.1:8787/>.

The **output folder** (-out, default campaigns/) is where every session is written, organised automatically:

campaigns/<EUT>/<test>/<YYYY-MM-DD_HHMMSS>/

Report.md	Obsidian report
results.xlsx	calculated tabular data (all attempts, incl. superseded)
session.json	machine-readable record
evidence/	
photos/	drop bench photos here
instrument-logs/	AZ export, DMM/PSU logs
screenshots/	instrument screen grabs
README.md	how photos correlate by timestamp

So material is grouped by **equipment-under-test** — **test** — **dated run**. Nothing to organise by hand.

2. Set up a test (the first screen)

Pick the test (e.g. *Protective Bonding Continuity*), then fill:

- **EUT:** product/version, serial, config ref, wiring-record ref.
- **Session:** tester, witness, location, title (auto if blank).

- **Connector:** bench-4wire (drives **both** instruments — Keysight sources the current, Rigol reads the drop). manual if no instruments (smart manual entry). Address is auto (leave blank); or a full VISA/LAN string to bypass discovery.
- **Test parameters:** main/small current, voltage limit, **source channel**, and the **bonding-point list**.

Keysight channel

CH1 = **6 V / 5 A** (the high-current output — use it for bonding). CH2/CH3 = 25 V / 1 A (more volts, less current). Keep the main current ≤ 5 A on CH1.

Editing the point list (auto-fill)

One point per line: ID | title | csa_mm2 | length_m | joints | small?

- **Progressive numbering:** leave the ID blank (start the line with |) and rows auto-number B-01, B-02, An explicit B-05 resets the counter so the next blank continues at B-06.
- **Repeated presets:** leave csa / length / joints / small blank and the row **repeats the value above** — set a preset once, override only where it changes.
- Leave csa+length blank entirely — smart manual entry (no computed expectation).
- Lines starting with # are comments.

Example — auto-numbered, preset repeated:

```
| CBP → DIN rail | 4 | 0.35 | 2 | no
| CBP → panel front | | 0.4 | | (repeats csa 4, joints 2, small no)
| CBP → panel rear | | 0.5 | |
```

The expected $m\Omega$ is computed from geometry ($17.2/CSA \times \text{length} + 5 m\Omega/\text{joint}$) and pre-filled, so the smart-entry field shows what you should see before measuring.

3. Run a point (the capture loop)

For each point the tool **forces the instrument settings** (channel, current, V-limit, output on) and shows a **colour status light**:

Colour	Meaning
 orange	powered & ready — no reading yet
 green	measuring OK — within expected +50 % and confirmed stable
 red	off limit — R above expected +50 % (or > 100 $m\Omega$)
 yellow	reading taken but stability not yet confirmed

Steps per point: 1. Press **r** (or *Read device*) — I comes from the Keysight, V (mV) from the Rigol; $R = V/I$ is computed live and the light updates. 2. Tick **Stable while flexing?** once confirmed (light goes green). 3. Optionally tick **Photo taken now** (its timestamp matches this record). 4. Press **Enter** (*Commit & next*) — the record is saved with a UTC timestamp and you advance to the next point.

Keyboard: Enter commit & next · r read · a re-apply settings.

Every record is **append-only** — committing never overwrites; it's persisted to session.json immediately.

4. Ambient — enter once, not per measurement

The **first step (ENV-00)** is the ambient/logger step: - Start the **AZ 88163** logger (press START 5 s until REC flashes), note its serial + sampling interval, take the overall setup photo, enter a baseline reading, and commit **once**. - **Measurement points do NOT ask for ambient again**. The logger runs on its own clock; you merge its full record at the end (below). So ambient is entered once.

5. Suspend on a failure, re-run the point, complete

If a point fails and needs rework (loose bond, re-seat, etc.):

1. **Pause** (button on the run page) — or just close the tool. State is saved; the source output is turned off. The session stays **resumable**.
2. Fix the hardware. Come back any time.
3. On the home screen, click **Resume last test** — everything is restored (setup, all captured records), no retyping, and you land on the next pending point.
4. **Re-run a failed point:** in the *Records* list, click **re-measure** on that point. It re-queues; when you commit, a **new attempt** is appended and the old one is kept (marked *superseded*) — full audit trail. The report shows the latest result per point plus a “superseded attempts” table.
5. **Add a point mid-run:** *Add a point* (same auto-fill rules) appends to the queue.
6. When all points read green (or are resolved), click **Finish**.

A session is only “closed” (not offered for resume) once you click **Finish**.

6. Finish — merge ambient, add photos, get the report

On the finish page:

1. **Import AZ ambient** — export the logger's file (its tool → *Convert to Excel*, a tab-delimited .csv) and upload it here. T/RH/Baro is merged into every record by

timestamp (the logger's UTC+1 clock is converted), and the logger is added to the equipment table. No retyping. *(The device stores data in a proprietary format, so its own exporter is needed to produce the file; the tool then ingests it.)*

2. **Photos & evidence** — drop bench photos into evidence/photos/, named with the record ID (e.g. Product_B-02_...). They correlate to records by the UTC capture timestamp shown in the report's evidence index. Screenshots → evidence/screenshots/, instrument logs → evidence/instrument-logs/.

The report (Report.md) and results.xlsx regenerate on every change, so they always reflect the latest state, including imported ambient and re-measurements.

7. What the record contains

- **Test equipment** table — instrument make/model/**serial**/firmware auto-captured from *IDN?, plus the AZ logger; calibration dates left ____ for you.
- **Results** — per point: conductor, expected $m\Omega$, I, V, measured R, stable, attempt #, verdict, observation. Plus a **superseded/re-measured** table.
- **Ambient & overall evidence** — baseline + merged logger summary.
- **Overall verdict, evidence index** (record → timestamp), and the vault record it feeds.